॥ वसुधैव कुटुम्बकम् ॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with "A" Grade

Name of the Student: Vatsal Postiwala **PRN:** 26070122509
Semester: III **Year:** AY 26-27

Subject Title: Operating Systems Lab	
EXPERIMENT No: 7	
TITLE: Loop Statements	**DoP**: 24-08-2026

Aim: Implement shell script to demonstrate loop statements.

Learning Outcome: 1. To understand the loop statements
2. To demonstrate shell script to implement loop statements using Linux Commands

Hardware/Software:

This experiment is performed using Ubuntu OS in a virtualized environment within Windows 11. The software used is:

- VMware Workstation 17 Player (Free Virtualization Tool)
- Ubuntu 22.04.1 LTS ISO file
- VMware Tools installed on the virtual machine

Problem Statement:

1. Until Loop – Sum of Numbers

Write a shell script using an until loop to accept a positive integer N and calculate the sum of all numbers from 1 to N.

```
Example for N = 5:
Sum = 15
```

2. Until Loop – Reverse Number Pattern

Write a shell script using an until loop to accept the number of rows N and print the following reverse number pattern.

```
Example for N = 5:
12345
1234
123
12
1
```

3. While Loop – Sum of Even and Odd Numbers

Write a shell script using a while loop to accept a positive integer N. Calculate and display separately the sum of all even numbers and the sum of all odd numbers from 1 to N.

```
Example for N = 10:
Sum of even numbers = 30
Sum of odd numbers = 25
```

4. While Loop – Number Pattern

Write a shell script using a while loop to accept the number of rows N and print the following pattern.

```
Example for N = 5:
1
22
333
4444
55555
```

5. Until Loop – Palindromic Number Pattern

Write a shell script using an until loop to accept the number of rows N and print the following pattern.

```
Example for N = 5:
1
121
12321
1234321
123454321
```

6. While Loop – Find the Largest and Smallest Digit

Write a shell script using a while loop to accept a positive integer from the user and find the largest digit and smallest digit present in the number.

```
Example:
Enter a number: 58342
Largest digit = 8
Smallest digit = 2
```

7. Until Loop – Check Whether a Number is Prime

Write a shell script using an until loop to accept a positive integer from the user and determine whether the given number is prime or not.

```
Example 1:
Enter a number: 17
17 is a Prime Number

Example 2:
Enter a number: 21
21 is Not a Prime Number
```

Theory:

The shell acts as an interface between the user and the OS services. It accepts human-readable commands from the user and converts them into a language that the kernel can understand. The

shell can be accessed using the terminal.

Shell accepts the command as input from the user and executes it. A shell can also take inputs as a file and execute them in case of executing a bunch of commands routinely. This helps to avoid the repetitive work of typing each command in the terminal. This file containing a bunch of commands is known as a shell script and is saved with the extension of .sh.

Three types of looping statements can be used in shell scripting. These include:

1. **while statement**
   A while loop is a control flow structure that allows to repeatedly execute a block of code as long as the condition specified is evaluated to be true
   The syntax is:
   while [condition]
   do
   #Statement blocks to be executed
   done
   Here, '[condition]' is the test condition to be evaluated before each iteration of the loop. If the condition is evaluated to be true, the statements in the body of the loop are executed. The iteration of the loop continues until the condition becomes false at which the control is transferred to the 'done' keyword signifying the termination of the while loop.

2. **for statement**
   A for loop is a control flow structure that allows repeatedly executing a block of code a specified number of times or for each item in a list.
   The syntax is:
   for i in list
   do
   #Statement blocks to be executed
   done
   Here, 'i' is the test iterator variable that takes the value of each item in the 'list'. The statements in the body of the loop are executed for each value of 'i'. The iteration of the loop continues until all the items in the list have been processed after which the control is transferred to the 'done' keyword signifying the termination of the for loop.

3. **until statement**
   An until statement is a control flow structure that allows repeatedly executing a block of code until a specified condition is true.
   The syntax is:
   until [condition]
   do
   #Statement blocks to be executed
   done
   Here, '[condition]' is the test condition that is evaluated before each iteration of the loop. If the condition is false, the statements inside the loop are executed. This process continues until the condition becomes true, at which point control is transferred to the statement following the done keyword.

   The until loop is similar to the while loop, except that the test condition is inverted. In other words, a while loop will continue to execute as long as the condition is true, while an until loop will continue to execute as long as the condition is false.

Note:
By default, the .sh file may not have executable permission. Hence, using the command chmod +x filename, the file is given executable permission.

#!/bin/bash signifies that the script should be executed using the bash shell

Algorithm:
For performing looping statements using shell scripting:

Program:
The contents of the shell script "assign6.sh" created in the virtualized environment of Ubuntu are:

Steps to Execute the Program:

1. Create a new file and make it executable using the chmod command
2. Edit the file using Text Editor and add the commands required for performing the desired looping statements.
3. Save and close the file. The file extension should be .sh
4. Run the script in the terminal using ./filename.sh
5. Based on the operations, provide the necessary inputs and the output is generated based on the evaluation/iteration of specified looping statements.

Output Screenshots:

1.

E7_1 × E7_2.sh ×

```
#!/bin/bash

echo "Enter a positive integer N:"
read N

sum=0
i=1

until [ $i -gt $N ]
do
    sum=$((sum + i))
    i=$((i + 1))
done

echo "Sum = $sum"
```

```
student@Ubuntu:~/OS$ bash E7_1.sh
Enter a positive integer N:
7
Sum = 28
```

2.

E7_1 E7_2.sh

```
#!/bin/bash

echo "Enter the number of rows N:"
read N

i=$N

until [ $i -lt 1 ]
do
    j=1

    until [ $j -gt $i ]
    do
        echo -n "$j"
        j=$((j + 1))
    done

    echo
    i=$((i - 1))
done
```

```
student@Ubuntu:~/OS$ bash E7_2.sh
Enter the number of rows N:
4
1234
123
12
1
```

3.

E7_3.sh E7_4.sh

```
#!/bin/bash

echo "Enter a positive integer N:"
read N

even_sum=0
odd_sum=0
i=1

while [ $i -le $N ]
do
    if [ $((i % 2)) -eq 0 ]
    then
        even_sum=$((even_sum + i))
    else
        odd_sum=$((odd_sum + i))
    fi

    i=$((i + 1))
done

echo "Sum of even numbers = $even_sum"
echo "Sum of odd numbers = $odd_sum"
```

```
student@Ubuntu:~/OS$ bash E7_3.sh
Enter a positive integer N:
5
Sum of even numbers = 6
Sum of odd numbers = 9
```

4.

E7_4.sh

```
#!/bin/bash

echo "Enter the number of rows N:"
read N

i=1

while [ $i -le $N ]
do
    j=1

    while [ $j -le $i ]
    do
        echo -n "$i"
        j=$((j + 1))
    done

    echo
    i=$((i + 1))
done
```

```
student@Ubuntu:~/OS$ bash E7_4.sh
Enter the number of rows N:
6
1
22
333
4444
55555
666666
```

5.

E7_5.sh

```
#!/bin/bash

echo "Enter the number of rows N:"
read N

i=1

until [ $i -gt $N ]
do
    j=1

    until [ $j -gt $i ]
    do
        echo -n "$j"
        j=$((j + 1))
    done

    j=$((i - 1))

    until [ $j -lt 1 ]
    do
        echo -n "$j"
        j=$((j - 1))
    done

    echo
    i=$((i + 1))
done
```

```
student@Ubuntu:~/OS$ bash E7_5.sh
Enter the number of rows N:
5
1
121
12321
1234321
123454321
```

6.

E7_6.sh

```
#!/bin/bash

echo "Enter a number:"
read N

largest=0
smallest=9

while [ $N -gt 0 ]
do
    digit=$((N % 10))

    if [ $digit -gt $largest ]
    then
        largest=$digit
    fi

    if [ $digit -lt $smallest ]
    then
        smallest=$digit
    fi

    N=$((N / 10))
done

echo "Largest digit = $largest"
echo "Smallest digit = $smallest"
```

```
student@Ubuntu:~/OS$ bash E7_6.sh
Enter a number:
9876
Largest digit = 9
Smallest digit = 6
```

7.

Open

E7_7.sh
~/OS

```
#!/bin/bash

echo "Enter a number:"
read N

if [ $N -lt 2 ]
then
    echo "$N is Not a Prime Number"
    exit
fi

i=2
flag=0

until [ $i -ge $N ]
do
    if [ $((N % i)) -eq 0 ]
    then
        flag=1
        break
    fi

    i=$((i + 1))
done

if [ $flag -eq 0 ]
then
    echo "$N is a Prime Number"
else
    echo "$N is Not a Prime Number"
fi
```

```
student@Ubuntu:~/OS$ bash E7_7.sh
Enter a number:
8
8 is Not a Prime Number
student@Ubuntu:~/OS$ bash E7_7.sh
Enter a number:
9
9 is Not a Prime Number
student@Ubuntu:~/OS$ bash E7_7.sh
Enter a number:
7
7 is a Prime Number
```

Conclusion:

I learned how to use while and until loops in shell scripting to solve different problems such as calculating sums, printing number patterns, finding the largest and smallest digits, and checking whether a number is prime. These programs helped me understand loops, conditions, arithmetic operations, and basic logic in shell scripting.

Assessment Schemes:

Attendance	Discipline	Short oral	Correctness of Lab Report	Timely completion of Lab Report	Total marks (10)	Signature of Teacher with Date